

API REST y servicios web en ASP.NET Core

Actividad: elaboración de un mapa mental

A partir de la lectura y de las fuentes bibliográficas proporcionadas, el estudiante deberá elaborar un **mapa mental sobre API REST y servicios web en ASP.NET Core**.

El mapa deberá demostrar la **comprensión y relación entre los conceptos**, evitando presentar únicamente definiciones aisladas.

1. Introducción a los servicios web

Los servicios web permiten que diferentes aplicaciones intercambien información y funcionalidades a través de una red. Una característica importante es que las aplicaciones que participan en la comunicación pueden estar desarrolladas con diferentes lenguajes de programación y tecnologías.

En las aplicaciones web modernas, HTTP constituye uno de los principales mecanismos de comunicación. Mediante este protocolo, un cliente puede realizar una solicitud a un servidor y recibir una respuesta que contiene información, resultados de una operación o información sobre un error (Fielding et al., 2022).

Los servicios web permiten establecer una separación entre diferentes componentes de un sistema. Por ejemplo, una aplicación puede encargarse de la interfaz que utiliza el usuario, mientras que otro componente proporciona los datos o funcionalidades requeridas.

Esta separación facilita la integración entre sistemas y permite que diferentes aplicaciones puedan utilizar los mismos servicios.

2. API: Application Programming Interface

API significa *Application Programming Interface* o **interfaz de programación de aplicaciones**.

Una API establece mecanismos mediante los cuales un software puede comunicarse con otro software. No necesariamente constituye una aplicación completa, sino que proporciona una interfaz mediante la cual otras aplicaciones pueden solicitar información o ejecutar determinadas operaciones.

Por ejemplo, una aplicación que necesita consultar información de productos podría realizar una solicitud a una API:

Aplicación → API → Base de datos

La API recibe la solicitud, procesa la operación correspondiente y devuelve una respuesta.

En el caso de ASP.NET Core, las API web permiten crear servicios que pueden ser utilizados por diferentes clientes, incluyendo aplicaciones web, aplicaciones móviles, aplicaciones de escritorio u otros sistemas (Microsoft, 2024a).

3. REST

REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos.

El concepto fue presentado por Roy Fielding en su tesis doctoral, donde definió REST como un estilo arquitectónico basado en un conjunto de restricciones aplicadas a sistemas distribuidos (Fielding, 2000).

REST **no es un lenguaje de programación ni un framework**. Es un enfoque arquitectónico que proporciona principios para diseñar servicios que utilizan una interfaz uniforme para interactuar con recursos.

Fielding (2000) señala que:

REST is an architectural style for distributed hypermedia systems.

Esta característica permite diseñar servicios en los que los recursos pueden identificarse mediante direcciones y manipularse mediante operaciones definidas por el protocolo utilizado.

En el contexto de las aplicaciones web, REST suele implementarse utilizando HTTP.

4. Recursos

Uno de los conceptos fundamentales de REST es el **recurso**.

Un recurso representa una entidad, objeto o conjunto de información que puede ser identificado mediante una URI.

Por ejemplo, en un sistema académico podrían existir recursos como:

/api/estudiantes

/api/cursos

/api/docentes

En este caso, /api/estudiantes podría representar una colección de estudiantes.

También puede identificarse un elemento específico:

/api/estudiantes/15

En este caso, la dirección identifica un estudiante concreto.

La utilización de recursos permite organizar una API de manera que las rutas representen aquello con lo que el sistema trabaja, mientras que los métodos HTTP indican qué operación se desea realizar.

5. URI y URL

Una **URI (Uniform Resource Identifier)** permite identificar un recurso.

En una API web, las URI suelen observarse mediante rutas como:

/api/productos

o:

/api/productos/15

La primera puede representar una colección de productos y la segunda un producto determinado.

En ASP.NET Core, estas rutas pueden establecerse mediante el sistema de **routing**. El enrutamiento determina qué acción del controlador debe ejecutarse cuando llega una determinada solicitud HTTP (Microsoft, 2024b).

6. Métodos HTTP

Las API REST utilizan los métodos HTTP para indicar qué operación se desea realizar sobre un recurso.

Los principales métodos utilizados son:

Método	Uso habitual
GET	Obtener información
POST	Crear un recurso
PUT	Actualizar o reemplazar un recurso
PATCH	Actualizar parcialmente un recurso
DELETE	Eliminar un recurso

Por ejemplo:

GET /api/productos

puede utilizarse para consultar los productos.

Mientras:

POST /api/productos

puede utilizarse para crear un nuevo producto.

Para trabajar con un producto específico:

GET /api/productos/15

PUT /api/productos/15

DELETE /api/productos/15

HTTP define la semántica de estos métodos y las reglas relacionadas con las solicitudes y respuestas (Fielding et al., 2022).

Por tanto, en una API REST la ruta y el método HTTP trabajan conjuntamente.

7. Solicitud HTTP

Cuando un cliente necesita utilizar una API, realiza una **solicitud HTTP**.

Una solicitud puede contener diferentes elementos:

- Método HTTP.
- URL.
- Encabezados.
- Parámetros.
- Cuerpo de la solicitud.

Por ejemplo:

POST /api/productos

podría acompañarse de un cuerpo que contenga los datos del nuevo producto.

El servidor recibe la solicitud y determina qué operación debe realizar.

HTTP establece la estructura y semántica de las solicitudes que permiten la comunicación entre clientes y servidores (Fielding et al., 2022).

8. Respuesta HTTP

Después de procesar una solicitud, el servidor genera una **respuesta HTTP**.

Una respuesta puede contener:

- Código de estado.
- Encabezados.
- Cuerpo de respuesta.
- Datos.
- Mensajes de error.

El proceso puede representarse como:

Por ejemplo, una API puede responder con información representada en JSON:

```
{  
  "id": 15,
```

```
"nombre": "Cuaderno",  
"precio": 25.50  
}
```

9. Códigos de estado HTTP

Los códigos de estado permiten informar al cliente sobre el resultado de una solicitud.

HTTP organiza los códigos de estado en cinco categorías principales:

- **1xx:** respuestas informativas.
- **2xx:** solicitudes procesadas correctamente.
- **3xx:** redirecciones.
- **4xx:** errores relacionados con la solicitud del cliente.
- **5xx:** errores relacionados con el servidor.

Algunos códigos habituales son:

Código	Significado
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Por ejemplo, 200 OK indica que la solicitud se procesó correctamente, mientras que 404 Not Found indica que no se encontró el recurso solicitado.

La clasificación y semántica de los códigos de estado están definidas por la especificación HTTP (Fielding et al., 2022).

10. JSON

JSON (JavaScript Object Notation) es un formato utilizado frecuentemente para representar e intercambiar información estructurada.

Un objeto puede representarse como:

```
{  
  "id": 15,  
  "nombre": "Cuaderno",  
  "precio": 25.50  
}
```

También es posible representar colecciones:

```
[  
  {  
    "id": 15,  
    "nombre": "Cuaderno"  
  },  
  {  
    "id": 16,  
    "nombre": "Carpeta"  
  }  
]
```

Su estructura permite representar información mediante propiedades y valores.

En una API ASP.NET Core, los objetos pueden convertirse a formatos adecuados para ser enviados como respuesta al cliente. Microsoft documenta el uso de JSON en las respuestas de las API web de ASP.NET Core (Microsoft, 2024a).

11. ASP.NET Core Web API

ASP.NET Core es un framework multiplataforma de Microsoft utilizado para desarrollar aplicaciones web y servicios.

ASP.NET Core permite desarrollar API web mediante diferentes enfoques. Entre ellos se encuentran las **Minimal APIs** y las API basadas en controladores (Microsoft, 2024a).

Microsoft (2024a) explica ASP.NET Core supports creating RESTful services using controllers or minimal APIs.

Las API basadas en controladores permiten organizar las operaciones mediante clases que reciben y procesan solicitudes HTTP.

Un ejemplo de un controlador podría ser:

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class ProductosController : ControllerBase
```

```
{
```

```
}
```

12. ControllerBase y Controller

En una API basada en controladores, las clases normalmente derivan de:

ControllerBase

ControllerBase proporciona funcionalidades relacionadas con el manejo de solicitudes HTTP y la generación de respuestas.

Por otro lado:

Controller

incorpora además funcionalidades relacionadas con las vistas utilizadas en aplicaciones MVC.

Por esta razón, en una aplicación que funciona principalmente como API es habitual utilizar ControllerBase, mientras que en una aplicación MVC tradicional se utiliza Controller (Microsoft, 2024a).

13. Controladores

Un **controlador** es una clase que contiene acciones capaces de procesar solicitudes.

Un controlador podría contener:

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class ProductosController : ControllerBase
```

```
{
```

```
    [HttpGet]
```

```
    public IActionResult Get()
```

```
    {
```

```
        return Ok();
```

```
    }
```

```
}
```

El atributo:

[ApiController]

proporciona comportamientos específicos para los controladores de API, incluyendo funcionalidades relacionadas con el enlace de parámetros y la validación de modelos (Microsoft, 2024a).

14. Routing

El **routing** o enrutamiento determina qué acción del controlador debe ejecutarse ante una solicitud.

Por ejemplo:

GET /api/productos

puede dirigirse a una acción encargada de obtener todos los productos.

Mientras:

GET /api/productos/10

puede dirigirse a una acción encargada de obtener el producto cuyo identificador es 10.

ASP.NET Core permite establecer las rutas mediante atributos:

[Route("api/[controller]")]

y asociar las acciones con métodos HTTP:

[HttpGet]

[HttpPost]

[HttpPut]

[HttpDelete]

Microsoft (2024b) explica que el sistema de routing de ASP.NET Core permite determinar qué acción de controlador debe procesar una solicitud HTTP.

15. Endpoint

Un **endpoint** es un punto de acceso de una API.

Puede entenderse mediante la combinación de:

Ruta + Método HTTP

Por ejemplo:

GET /api/productos

representa un endpoint para consultar productos.

Mientras:

POST /api/productos

representa otro endpoint destinado a crear productos.

Aunque ambos utilizan la misma ruta, el método HTTP permite diferenciar las operaciones.

Por ello, al estudiar una API es necesario analizar:

- Ruta.
- Método HTTP.
- Parámetros.
- Datos enviados.
- Respuesta.
- Código de estado.

16. Modelos

Los modelos representan la información con la que trabaja la aplicación.

En C#:

```
public class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public decimal Precio { get; set; }
    public int Stock { get; set; }
}
```

Los modelos pueden representar datos que llegan desde el cliente o información que será enviada como respuesta.

En aplicaciones que utilizan bases de datos, estos modelos pueden estar relacionados con entidades gestionadas mediante Entity Framework Core.

17. Entity Framework Core

Entity Framework Core (EF Core) es una tecnología de acceso a datos desarrollada por Microsoft para aplicaciones .NET.

Permite trabajar con bases de datos utilizando objetos y clases de .NET.

En una API puede utilizarse para:

- Consultar información.

- Insertar registros.
- Modificar registros.
- Eliminar registros.

De esta forma, la API puede actuar como intermediaria entre el cliente y la base de datos.

18. DbContext

El DbContext representa una sesión de trabajo con la base de datos y permite realizar operaciones mediante Entity Framework Core.

Por ejemplo, una aplicación puede tener:

```
public class AppDbContext : DbContext
{
    public DbSet<Producto> Productos { get; set; }
}
```

En este ejemplo, DbSet<Producto> representa el conjunto de productos que será gestionado mediante Entity Framework Core.

El controlador puede utilizar el contexto para consultar o modificar información.

19. Inyección de dependencias

ASP.NET Core incorpora un sistema de **inyección de dependencias (Dependency Injection)**.

Este mecanismo permite proporcionar a una clase las dependencias que necesita sin que tenga que crearlas directamente.

Por ejemplo, un controlador puede recibir un contexto mediante su constructor:

```
private readonly AppDbContext _context;

public ProductosController(AppDbContext context)
{
    _context = context;
}
```

Microsoft documenta la inyección de dependencias como una funcionalidad integrada en ASP.NET Core que permite registrar y proporcionar servicios a los componentes de la aplicación (Microsoft, 2024c).

20. API y base de datos

Una API normalmente funciona como una capa intermedia entre el cliente y la base de datos.

Tomando en cuenta un modelo simplificado, cuando el cliente realiza una solicitud, el controlador procesa la petición y puede utilizar servicios de acceso a datos para obtener o modificar información.

El cliente, por tanto, no necesita conocer directamente cómo está organizada la base de datos.

Esta separación permite modificar la implementación interna del sistema sin cambiar necesariamente la forma en que los consumidores utilizan la API.

21. Consumo de una API

Una API puede ser consumida por diferentes tipos de clientes:

- Aplicaciones web.
- Aplicaciones móviles.
- Aplicaciones de escritorio.
- Otros servicios web.
- Sistemas externos.

El consumidor de la API necesita conocer la interfaz que esta proporciona: sus endpoints, métodos HTTP, parámetros, datos requeridos y respuestas.

De esta manera, diferentes aplicaciones pueden utilizar un mismo servicio sin necesidad de conocer toda su implementación interna.

22. OpenAPI

OpenAPI es una especificación utilizada para describir API HTTP.

Una descripción OpenAPI puede incluir información sobre:

- Rutas.
- Métodos HTTP.
- Parámetros.
- Solicitudes.
- Respuestas.
- Códigos de estado.
- Esquemas de datos.

La especificación permite que una API pueda ser descrita de una manera estructurada y comprensible para herramientas y desarrolladores (OpenAPI Initiative, 2025).

Por ejemplo, una API podría documentar:

GET /api/productos

POST /api/productos

GET /api/productos/{id}

PUT /api/productos/{id}

DELETE /api/productos/{id}

23. Swagger

Swagger es un conjunto de herramientas asociado al ecosistema OpenAPI.

Una de las herramientas más conocidas es **Swagger UI**, que permite visualizar la documentación de una API y, dependiendo de su configuración, probar sus endpoints desde una interfaz web.

Es importante distinguir:

OpenAPI → especificación

Swagger → herramientas

Microsoft documenta el uso de Swagger/OpenAPI para generar páginas de ayuda y documentación interactiva para API web desarrolladas con ASP.NET Core (Microsoft, 2024d).

24. Seguridad en las API

La seguridad constituye un aspecto importante en el desarrollo de API.

Entre los conceptos relacionados con la seguridad se encuentran:

- **Autenticación:** permite determinar quién está realizando una solicitud.
- **Autorización:** determina qué acciones puede realizar un usuario o aplicación.
- **HTTPS:** protege la comunicación entre cliente y servidor mediante cifrado.
- **Validación:** permite verificar que los datos recibidos cumplen las condiciones esperadas.
- **Control de acceso:** limita el acceso a determinados recursos u operaciones.

Es importante diferenciar autenticación y autorización. Una aplicación puede identificar correctamente a un usuario, pero esto no significa que dicho usuario tenga permiso para realizar todas las operaciones disponibles.

La seguridad debe considerarse desde el diseño de la API y no únicamente después de implementar sus funcionalidades.

25. Ventajas de las API REST con ASP.NET Core

El desarrollo de API REST mediante ASP.NET Core permite implementar servicios que pueden ser utilizados por diferentes tipos de clientes.

Entre sus características se encuentran:

- Utilización de HTTP.
- Organización mediante recursos.
- Uso de métodos HTTP.
- Intercambio de información mediante JSON.
- Separación entre cliente y servidor.
- Integración con bases de datos.
- Utilización de Entity Framework Core.
- Inyección de dependencias.
- Validación de modelos.
- Implementación de autenticación y autorización.
- Documentación mediante OpenAPI.
- Integración con diferentes aplicaciones.

ASP.NET Core proporciona diferentes mecanismos para implementar API, incluyendo controladores, routing, modelos, respuestas HTTP, inyección de dependencias y mecanismos de validación (Microsoft, 2024a).

26. Funcionamiento integral de una API REST

Este flujo permite observar que una API REST no consiste únicamente en crear un controlador.

En su funcionamiento intervienen diferentes elementos: **cliente, HTTP, recursos, rutas, endpoints, controladores, modelos, servicios, acceso a datos, base de datos y respuestas.**

La relación entre estos elementos permite construir servicios que pueden ser utilizados por diferentes aplicaciones.

Bibliografía

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. University of California, Irvine. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Fielding, R. T., Nottingham, M., & Reschke, J. (2022). *HTTP semantics (RFC 9110)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>

Microsoft. (2024a). *API web de ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/es-es/aspnet/core/web-api/?view=aspnetcore-8.0>

Microsoft. (2024b). *Enrutamiento a acciones de controlador en ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/es-es/aspnet/core/mvc/controllers/routing?view=aspnetcore-8.0>

Microsoft. (2024c). *Inyección de dependencias en ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/es-es/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-8.0>

Microsoft. (2024d). *Creación de páginas de ayuda de API web con Swagger/OpenAPI*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/web-api-help-pages-using-swagger?view=aspnetcore-8.0>

OpenAPI Initiative. (2025). *OpenAPI specification*. Linux Foundation. <https://spec.openapis.org/oas/latest.html>

Mozilla Developer Network. (s. f.). *HTTP response status codes*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>